

AI Credit Risk & Loan Approval Engine

Complete project guide: what was built, how, why, and how to explain it in an interview

1. Problem statement

A lender receives a loan application and must decide, at that moment, whether to lend. If it says yes to someone who later stops paying, it loses most of the money still owed. If it says no to someone who would have repaid, it loses the interest it would have earned and a customer. The decision has to be made with only the information available on the application day: income, debts, credit-bureau history, the loan amount and term. Nothing that happens later may be used.

The goal of this project is to build that decision engine end to end on real data: **LendingClub**, a US peer-to-peer lender, published every loan it issued between 2007 and 2018 (2,260,701 loans, 151 columns). For each loan we know what was known at application time and what eventually happened (repaid or charged off).

What the engine must output for every application

- **Probability of default (PD)**: a calibrated number between 0 and 1. "PD = 20%" must mean that about 20 of 100 such applicants really default.
- **Risk band** R1 (safest) to R7: a short label, like a credit grade.
- **Decision**: APPROVE, REVIEW (a person looks at it) or DECLINE, plus REFER when the applicant is outside what the model has ever seen.
- **Expected loss in dollars**: $PD \times \text{amount owed at default} \times \text{share not recovered}$.
- **Reasons**: the main factors that raised or lowered this applicant's risk, in plain language (needed for adverse-action notices in US lending).

Constraints that make it a real-world problem, not a Kaggle exercise

- **No leakage**: 42 of the 151 columns are only known after the loan starts (payments, recoveries, last FICO...). They must be removed or the model would look brilliant and be useless.
- **Out-of-time testing**: the model is trained on older loans and tested on newer ones, as it would be used.
- **No lender shortcuts**: LendingClub's own grade and interest rate are its risk opinion; the deployable model must not use them.
- **Imbalanced target**: only ~20% of loans default, so accuracy is a misleading metric.
- **Deployable**: one reproducible package, a public web app and an API, with input checks and guardrails.

Final result in one line: a monotonic, calibrated LightGBM model on 146 application-time features reaches **ROC-AUC 0.746** on a sealed test set of **212,801** later loans (LendingClub's own interest rate: 0.715), approves 66.8% of applicants at an 11.6% default rate (vs 20.1% overall), and runs live at credit-risk-engine-dhib.onrender.com.

2. Why I chose this project

Credit risk is one of the oldest and most valuable uses of machine learning. Every bank, card issuer, fintech and buy-now-pay-later company runs a probability-of-default model. It is a problem where a small improvement in ranking is worth millions, and where mistakes are expensive and regulated. Working on it shows the skills companies actually pay for: careful data work, honest evaluation, explainability and deployment, not just fitting a model.

The data is real and messy. LendingClub's file has 2.26 million rows and 151 columns, with missing values that mean different things, columns that leak the future, free-text fields, dates stored as text and a target that has to be defined from nine different loan statuses. That makes it a much better learning project than a clean toy dataset, because most of the real work in industry is exactly this kind of cleaning and auditing.

It covers the whole lifecycle in one project. Data understanding → leakage audit → preprocessing pipeline → model comparison from a dummy baseline up to gradient boosting → calibration → explainability (SHAP) → fairness check → business decision layer → packaging → final test → web deployment → responding to an external audit. Very few portfolio projects go past "I trained XGBoost and got an AUC".

It has a clear business answer. At the end the question is not "what is the accuracy" but "how many people do we approve, what default rate do we accept, and how much money do we lose". That connects machine learning to decisions, which is what interviewers in fintech, banking and analytics roles want to hear about.

It is explainable by design. In lending you must tell a declined applicant why. That forces the use of monotonic constraints (higher FICO can never increase risk) and SHAP reason codes, which are good topics to discuss in interviews because they show you understand that a model is used by people, not only scored by metrics.

Remember: credit risk = high business value + messy real data + full ML lifecycle + explainability. That is why it is a strong resume project.

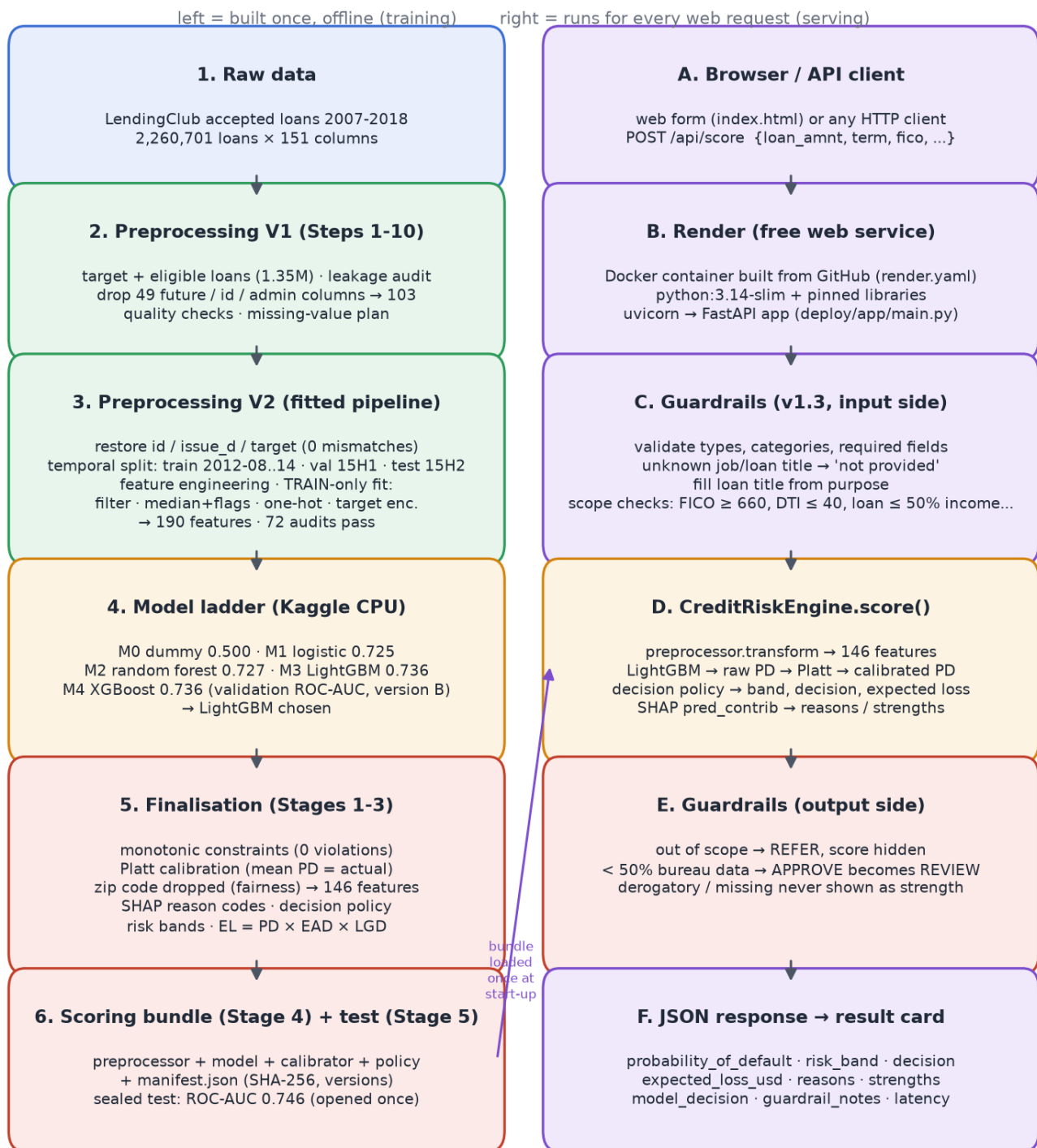
3. What this project teaches

Skill	Where it appears in the project
Problem framing	Defining the target from 9 loan statuses; deciding which loans have a final outcome; choosing what the model may know at decision time.
Data leakage	Column-by-column audit: 94 application-time vs 42 post-loan columns; the single-feature AUC check.
Reproducible preprocessing	One scikit-learn Pipeline fitted on train only, saved with joblib, tested in a fresh process.
Time-based validation	Train 2012-08..2014-12, validate 2015 H1, test 2015 H2; why random splits overstate quality.
Encoding categorical data	One-hot with rare-category grouping; cross-fitted target encoding for zip / job title.
Missing values	Median imputation + 'was missing' flags, because missing often means 'never happened'.
Model selection	A ladder M0 → M4 where each model must beat the previous one; random search with early stopping.
Imbalanced classification metrics	ROC-AUC, PR-AUC, KS, Brier, log loss, calibration gap, ECE, and why not accuracy.
Probability calibration	Platt scaling vs isotonic regression, chosen by out-of-fold Brier score.
Domain constraints	Monotonic constraints in LightGBM; verified with a sweep (3,709 violations → 0).
Explainability	SHAP values from LightGBM, grouped into concepts, turned into plain-language reason codes.
Fairness	Zip-code ablation (proxy for where people live); state-level calibration check.
Business decisions	Risk bands, approve / review / decline thresholds from business targets, expected loss $PD \times EAD \times LGD$.
Honest evaluation	Test set sealed in code, criteria committed to git before opening it, opened exactly once.
MLOps basics	Checksummed model bundle, version pinning, fresh-session tests, FastAPI, Docker, Render, auto-deploy.
Working with constraints	16 GB laptop: memory-safe chunked reads; heavy training moved to free Kaggle CPU kernels.
Handling critique	Two independent black-box audits → guardrails v1.1-v1.3 without retraining the frozen model.

Remember: the biggest lesson is that the model is the smallest part. Defining the target, removing leakage, testing honestly and turning a probability into a safe decision is most of the work.

4. Architecture: how the whole system works

End-to-end architecture: from raw CSV to a live decision



One GitHub repository holds both sides. A push to main makes Render rebuild the Docker image and redeploy. Training data never goes into the container: only the ~10 MB bundle (checksummed) and the app code.

Reading the architecture

Left side (offline, done once). The raw CSV is cleaned and audited in the V1 notebook (Steps 1-10), then the V2 code builds a single fitted preprocessing pipeline and the model-ready Parquet files. Five model families are trained and compared (heavy ones on Kaggle). The winner is finalised in stages: monotonic constraints and calibration, reason codes and fairness, the decision policy, packaging into a bundle, and the one-time test.

The bundle is the bridge. Four files (fitted preprocessor, LightGBM model, Platt calibrator, decision policy) and a manifest with their SHA-256 checksums and library versions. The web app loads exactly these files; if a file is changed or scikit-learn is the wrong version, the engine refuses to start. That guarantees the live app scores exactly like the model that was tested.

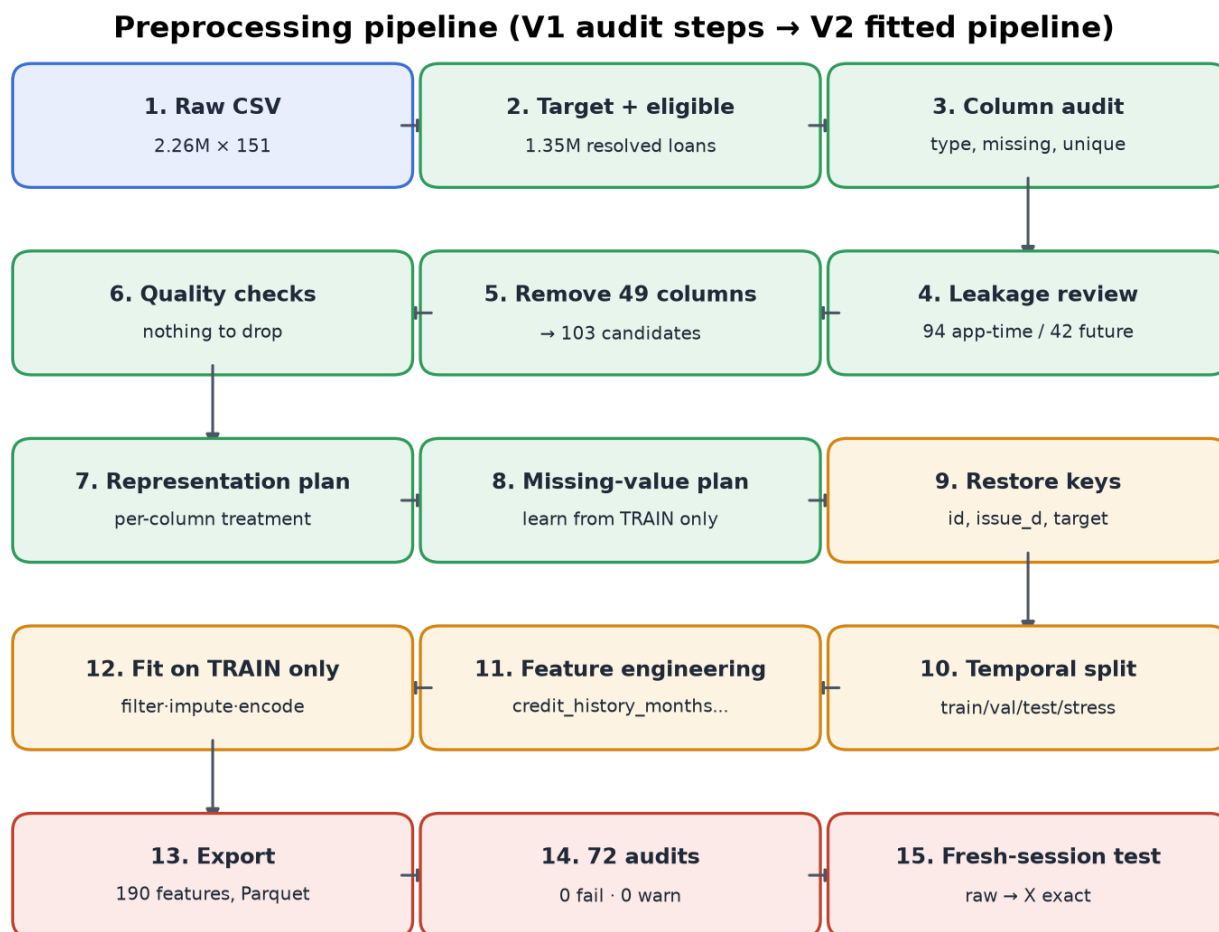
Right side (online, every request). A browser or API client sends JSON to Render. Render runs our Docker container: uvicorn (the web server) passes the request to the FastAPI app, which validates the input, applies the input guardrails, and calls *CreditRiskEngine.score()*. The engine runs the same preprocessing pipeline, gets the raw PD from LightGBM, calibrates it, applies the policy (band, decision, expected loss), computes SHAP reasons, then applies the output guardrails and returns JSON that the web page shows as a result card.

One request, step by step

- Form sends `{loan_amnt: 12000, term: 36, annual_inc: 92500, dti: 18.4, fico_range_low: 665, ...}`.
- Validation: required fields present, categories known, numbers valid, dates not in the future.
- Scope check: FICO ≥ 660 , DTI ≤ 40 , loan \$1k-\$35k and $\leq 50\%$ of income, etc. If not $\rightarrow$ REFER, no score shown.
- Preprocessor: credit_history_months from the credit-line date, medians for missing fields, one-hot and target encoding $\rightarrow$ 146 numbers in the exact training order.
- LightGBM (1,931 trees) $\rightarrow$ raw PD $\rightarrow$ Platt calibration $\rightarrow$ calibrated PD, e.g. 19.5%.
- Policy: $19.5\% \leq 22.5\% \rightarrow$ APPROVE, band R4; expected loss = $0.195 \times 12,000 \times 0.573 \times 0.887 \approx \$1,190$.
- SHAP: top 4 risk reasons and 2 strengths; JSON returned in ~ 0.7 s on the free server.

5. Preprocessing (in detail)

Preprocessing turned 2.26 million raw rows with 151 columns into four clean, model-ready datasets with 190 numeric features, plus one saved pipeline that can transform a brand-new application in exactly the same way. It happened in two parts: the **V1 notebook** (Steps 1-10: understanding, target, leakage audit, quality) and **Preprocessing V2** (Python modules in *src/preprocessing_v2/*: restore keys, temporal split, feature engineering, a fitted pipeline, export, 72 audits and a fresh-session test). V2 replaced V1's later steps after a review found five problems.



blue = data · green = V1 notebook (Steps 1-10, audits) · orange = V2 build · red = outputs and checks

Part A: understanding the data (V1 notebook, Steps 1-10)

Step 1. Load the data and define the prediction point

The raw file *accepted_2007_to_2018Q4.csv* has 2,260,701 loans and 151 columns. Before touching any column we fixed the ground rule for the whole project: the model scores a **new application**, so it may only use information that exists at or before the moment the lender decides. This single rule drives every later choice, especially the leakage audit.

Why it matters: a model trained with information from the future (for example the total amount repaid) will score almost perfectly in a notebook and fail completely in real use, because that information does not exist when the application arrives. Fixing the prediction point first avoids that trap.

Step 2. Investigate loan_status

The column *loan_status* says what happened to each loan. It has 9 distinct values plus 33 missing ones: Fully Paid, Charged Off, Current, Late (31-120 days), Late (16-30 days), In Grace Period, Default, and two 'Does not meet the credit policy' variants (Fully Paid / Charged Off) for older loans issued under a previous policy.

Understanding these values is necessary to define the target correctly. 'Current' or 'Late' loans have no final outcome yet: a late loan may still recover and a current loan may still default. Treating them as good or bad would be guessing.

Step 3. Define the target

target = 1 (default): Charged Off, Default, and 'Does not meet the credit policy: Charged Off'. **target = 0 (repaid):** Fully Paid and 'Does not meet the credit policy: Fully Paid'. Current, Late, In Grace Period and missing statuses get no target.

Charged Off means LendingClub gave up collecting (usually after 120+ days late), which is the standard meaning of default in consumer lending. Only 1 loan was in the 'Default' status at the snapshot, so in practice the target is 'charged off vs fully paid'.

Step 4. Define the eligible population

Only loans with a final outcome are kept: **1,348,099 resolved loans** (59.6% of all rows); 912,602 unresolved loans are dropped. The class balance is 1,078,739 repaid (80.0%) vs 269,360 defaults (20.0%).

A 20% default rate is imbalanced but not extreme. It means a model that always says 'repaid' is already 80% accurate, which is why accuracy is never used as the main metric in this project (see the metrics chapter).

Step 5. Column audit

For all 152 columns (151 + target) we recorded the data type, missing count and percentage, number of unique values and sample values. 40 columns are at least 80% missing and 7 are constant or single-valued. The audit table is saved in *reports/preprocessing_v1_audits/accepted_column_audit.csv*.

The audit is the map for every later decision: which columns can be numbers, which are categories, which are dates stored as text, and which are nearly empty. Many 'nearly empty' columns turned out to be fields that LendingClub only started collecting in later years, which became important for the temporal split.

Step 6. Application-time vs future-time audit (the leakage review)

Every column was put in exactly one class by asking: *would we know this value when the application arrives?* Result: **94 application-time features** (income, DTI, FICO, credit-bureau history, loan amount, purpose, state...), **42 post-prediction columns** only known after the loan starts (payments received, recoveries, outstanding principal, last payment date, last FICO, last credit pull, hardship plans, debt settlement, funded amounts, issue date), 4 LendingClub-derived columns kept for review (int_rate, installment, grade, sub_grade), 4 text / high-cardinality fields, 1 historical date, 2 target, 2 identifier and 3 administrative columns.

This is the most important step of the whole project. For example *recoveries* is non-zero almost only for defaulted loans, and *last_fico_range_high* drops sharply after a borrower stops paying; either would give a fake near-perfect model. The four LendingClub-derived columns are legal at decision time but are LendingClub's own

risk opinion, so later every model is trained with them (version A, benchmark) and without them (version B, deployable).

Step 7. Remove target, leakage, identifiers and administrative columns

49 columns were dropped: 2 target, 2 identifiers (id, member_id), 3 administrative (url, policy_code, initial_list_status) and the 42 post-prediction columns. The result is **1,348,099 rows × 103 candidate columns**, and checks confirmed that no target, id or future column remained.

Identifiers are dropped because a loan number carries no information about the borrower and can only add noise or leak ordering. Administrative fields describe how LendingClub listed the loan, not the borrower's risk.

Step 8. Data-quality investigation

We checked missingness, all-missing columns, constant columns, duplicate column names, columns with identical content, duplicate rows, infinite values, numeric ranges and category counts. Nothing needed to be removed at this stage, so the dataset stayed 1,348,099 × 103.

Doing the checks and finding nothing is still valuable: it is documented evidence (saved audit CSVs) that the data is sound before modelling, which is exactly what a reviewer or interviewer will ask about.

Step 9. Feature semantics and representation plan

Each column was labelled numeric, categorical, date, free text or LendingClub-derived, with a planned treatment. Key decisions: *desc* (free-text loan description) is left out of the first version because it needs a separate NLP pipeline; *earliest_cr_line* (a date like 'Aug-2003') becomes a number of months; *emp_title*, *title* and *zip_code* need an encoder for high-cardinality categories.

Planning the representation before coding avoids accidental mistakes such as one-hot encoding a date (which V1 later did by accident, creating 733 columns from one date).

Step 10. Missing-value strategy

Columns were grouped by how much they are missing, with a strategy for each. The rule carried into V2: every value used to fill gaps (medians, category labels) must be learned from **training data only**, never from validation or test.

Learning medians from all the data would let information from the future test period leak into training. It is a small leak, but in credit risk the test must simulate the future honestly.

Why V1 Steps 11-16 were replaced by Preprocessing V2

V1 originally finished with a random 80/10/10 split and 1,943 columns. A review found five problems: (1) the labels were never exported, so no model could load them; (2) *earliest_cr_line* was one-hot encoded into 733 columns instead of one number; (3) *zip_code* was one-hot encoded into 936 columns; (4) the fitted encoders and medians were never saved, so a new application could not be scored; (5) the random split mixes years, while credit models must be tested on later loans.

Instead of patching, V2 rebuilt Steps 11-16 as tested Python modules and kept all V1 outputs untouched as a backup. Being able to explain why you threw work away and redid it properly is a good interview story.

Part B: Preprocessing V2 (src/preprocessing_v2/, run once with run_preprocessing_v2.py)

Step 11. Restore id, issue date and target safely

The V1 output had no id, issue date or target (Step 7 removed them on purpose). V2 re-reads the raw CSV in its original order with the same target mapping (1,348,099 rows) and re-attaches them by position, but only after a safety check: 12 shared columns (loan_amnt, int_rate, annual_inc, dti, revol_util, fico_range_low, term, grade, emp_title, title, zip_code, earliest_cr_line) are compared row by row.

Result: 0 mismatches, ids unique, every issue date parses. If even one row had differed, the run would have stopped. Attaching labels by position without such a check is a classic silent bug that misaligns X and y.

Step 12. Temporal split by issue month

Train = loans issued 2012-08 to 2014-12 (388,124 loans, 17.24% default); validation = 2015 H1 (162,745, 20.35%); test = 2015 H2 (212,801, 20.06%); stress = 2016-2018 (518,744, 22.42%, optional). Loans before Aug-2012 (65,685) are excluded.

Why start in Aug-2012: 33 credit-bureau fields (tot_cur_bal, num_*, mo_sin_*) only start being filled in Mar-Aug 2012; earlier loans would teach the model that 'missing' means 'old loan'. Why not test on 2016+: at the 2018 snapshot only 67.5% of 2016 loans (38% of 2017, 11% of 2018) had finished, and the finished ones are mostly early defaults and early payoffs, so their default rate is distorted. Every split still has at least 33k defaults, so no resampling is needed.

Step 13. Feature engineering (row by row, nothing learned from data)

credit_history_months = (application year – first-credit-line year) × 12 + (application month – first-credit-line month). It replaces the 733 date columns with one number (train range 36-842 months, median 177).

term_months: '36 months' → 36. **emp_length_years**: '< 1 year' → 0 ... '10+ years' → 10. Text categories are stripped and zip_code / emp_title / title lowercased so 'Teacher' and 'teacher' are the same.

These transformations are stateless (nothing is learned), and the exact function lives inside the saved pipeline, so a new application goes through identical steps. Using the application month (not today's date) for credit history is what keeps the feature honest for historical loans.

Step 14. Fitted pipeline, trained on TRAIN only

One scikit-learn Pipeline: (a) a **train-only column filter** drops 31 columns that are >95% missing or constant in train (joint-application fields, second-applicant fields, and 14 bureau fields LendingClub only began reporting in late 2015); (b) a **ColumnTransformer** with three branches: 62 numeric columns get the train median plus a 0/1 'was missing' flag; 6 categorical columns are one-hot encoded with categories under 500 training rows grouped as 'infrequent'; 3 high-cardinality columns (zip_code, emp_title, title) are **target encoded**; (c) 17 duplicate 'was missing' flags are dropped. Result: 103 raw columns → **190 numeric features**.

Target encoding replaces each category with its smoothed training default rate (e.g. job title 'nurse' → 15.2%). To avoid leakage, training rows get 5-fold **out-of-fold** values (a row never sees its own label); validation and test use the train-only mapping; unseen categories get the overall train default rate (17.24%). The **missing flags** matter because 'missing' often means 'never happened' (e.g. months since last delinquency is missing for people who were never delinquent), which is useful information.

Step 15. Export

Each split is transformed (validation / test / stress in 100k-row chunks to save memory), written to Parquet, then read back and checked. X_<split>.parquet holds the 190 features; y_<split>.parquet holds id, issue_d and target in exactly the same row order. The whole fitted pipeline is saved as one file, *preprocessor_v2.joblib*, and *metadata_v2.json* records split windows, feature names, feature groups and library versions.

Saving the fitted pipeline is what makes deployment possible: the web app loads the same object and transforms a raw application identically. Parquet keeps types and is ~10× smaller and faster than CSV.

Step 16. 72 audits and a fresh-session test

72 automated checks in 7 groups, all passed: target alignment (20), temporal ordering (8), leakage (9), data hygiene (17: 0 NaN, 0 infinite, 0 non-numeric, 0 duplicate columns in every split), consistency (9), persistence and backups (4), engineering / readiness (5). The highest single-feature AUC is 0.674 (int_rate), below the 0.80 review flag that would suggest leakage.

The fresh-session test runs in a separate Python process: it loads the saved pipeline, takes the raw CSV rows of 5 test loans and reproduces the exported features exactly (max difference 0.0), and transforms a hand-written new application with an unseen zip code and missing fields without any NaN. This proves the pipeline works outside the notebook.

Remember: leakage audit + temporal split + fit on train only + saved pipeline + audits. If asked 'how do you know there is no leakage?', mention the 42 removed columns, out-of-fold target encoding, the single-feature AUC check (max 0.674) and the 72 audits.

6. The models, step by step (M0 → M4, then the final model)

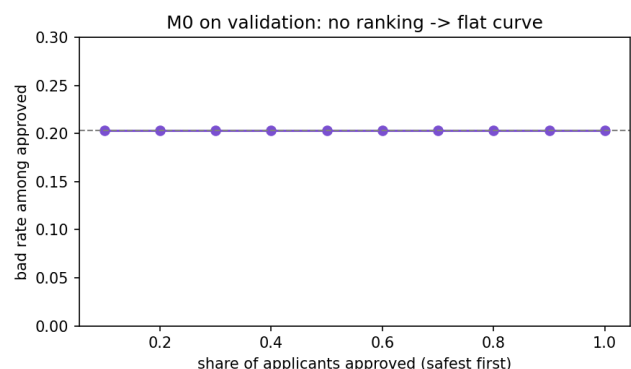
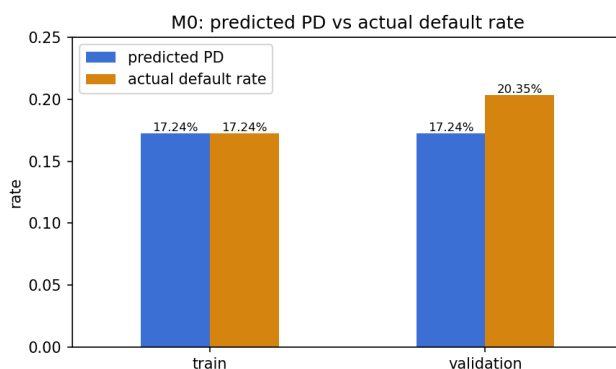
The models were added in order of complexity, and each one had to beat the previous one on the validation set (2015 H1). Every model was trained in two versions: **A** uses LendingClub's grade, sub-grade, interest rate and instalment; **B** does not. LendingClub set those values from its own risk model after reviewing the application, so a real lender building its own engine would not have them. **B is the deployable version**; A is a benchmark that shows how much of LendingClub's opinion our features can rebuild. All numbers below are validation unless marked test.

Model	Version A ROC-AUC	Version B ROC-AUC	B Brier	Verdict
M0 Dummy	0.500	0.500	0.1631	floor
M1 Logistic Regression	0.7335	0.7252	0.1449	strong, simple baseline
M2 Random Forest	0.7345	0.7270	0.1467	no real gain (+0.002)
M3 LightGBM	0.7414	0.7362	0.1426	clear gain (+0.011) → chosen
M4 XGBoost	0.7412	0.7356	0.1427	tie, confirms M3

M0: Dummy baseline

What it is: a model that ignores every feature and predicts the same PD for everyone: the training default rate, 17.24% (scikit-learn DummyClassifier, strategy 'prior'). **Why build it:** it is the floor. Any real model must beat it, and its scores show what 'zero intelligence' looks like for each metric.

Results: ROC-AUC exactly 0.500 (it cannot rank anyone), KS 0, Brier 0.1631 on validation. Its calibration gap is -3.1 points because the default rate rose from 17.2% in train to 20.4% in validation: the first sign of **drift over time**, which is why calibration had to be redone on newer data later.

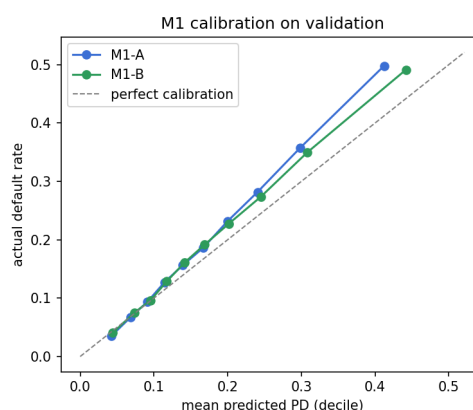
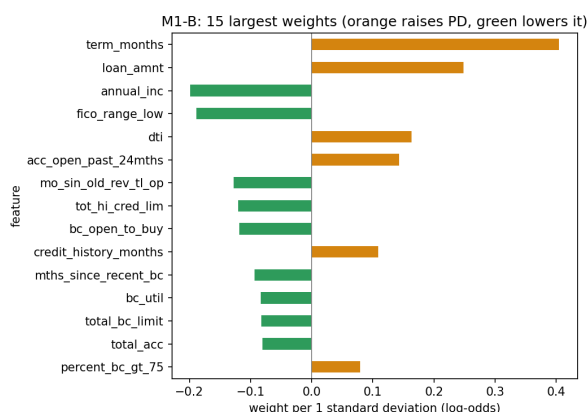


M1: Logistic Regression

How it works: a linear score $z = b_0 + b_1x_1 + \dots + b_nx_n$ passed through the sigmoid PD = $1 / (1 + e^{-z})$. Each coefficient is the change in log-odds of default for one unit of a feature. It is the classic credit-scorecard model: fast, stable and easy to explain.

Preparation: 26 skewed columns (income, balances, limits) were log-transformed with log1p, then everything was standardised (StandardScaler) so the coefficients are comparable and the solver converges. fico_range_high was dropped because it is always fico_range_low + 4 (perfectly collinear). **Tuning:** L2 regularisation strength C from {0.0001...1}, with or without class_weight='balanced', chosen by validation ROC-AUC. Chosen: log+scale, C = 0.001 (B), no class weights.

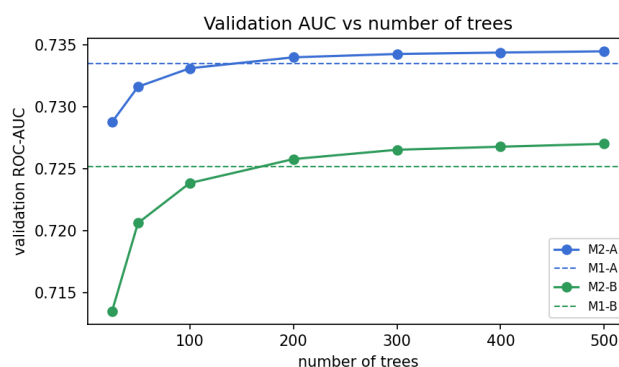
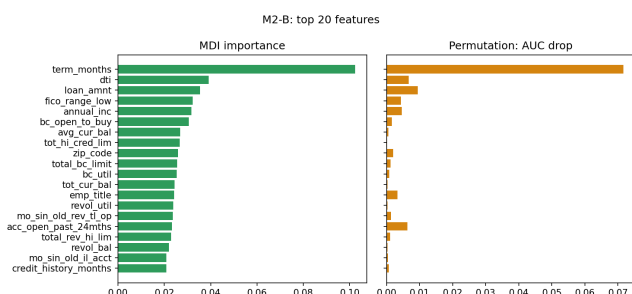
Results: validation ROC-AUC 0.7252 (B) / 0.7335 (A), Brier 0.1449. A big jump from 0.5, which already shows the features carry real signal. Interesting fact for interviews: at a 0.5 threshold it is 80% accurate but catches only about 3% of defaulters, which is exactly why accuracy is a bad metric here.



M2: Random Forest

How it works: many decision trees, each trained on a random half of the rows and a random subset of features; the forest's PD is the average of the trees' votes. It captures non-linear effects and interactions that logistic regression cannot. **Tuning:** min_samples_leaf {20, 50, 100} and max_features {sqrt, 0.3}, 500 trees. Run on a Kaggle CPU kernel because the laptop ran out of memory.

Results: validation ROC-AUC 0.7270 (B), only +0.002 over logistic regression, and a worse Brier (0.1467). Its train AUC (0.855) is far above validation, a sign of overfitting: deep trees memorise. Conclusion: non-linearity alone did not help much; boosting was needed.

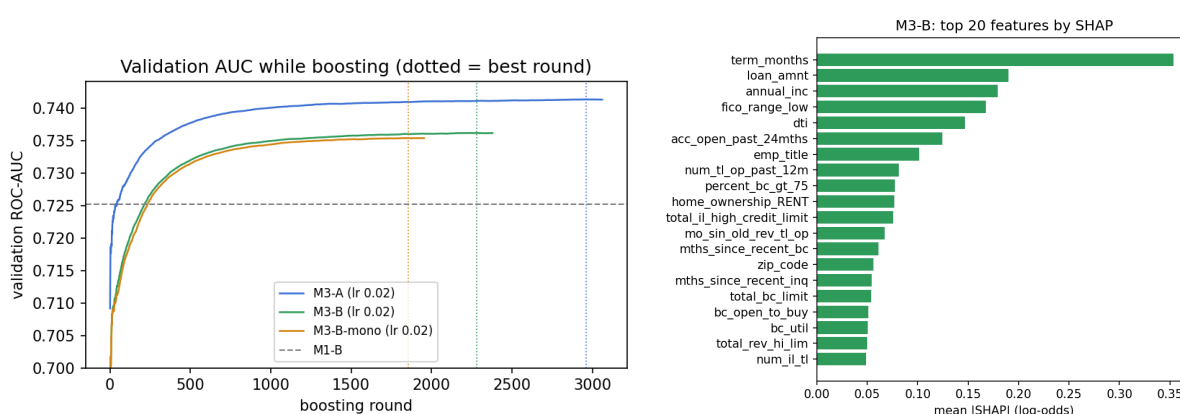


M3: LightGBM (gradient boosting) — the chosen model

How gradient boosting works: trees are built one after another; each new small tree is fitted to the errors (gradients of the log loss) of all previous trees, and its prediction is added with a small weight (the learning rate). After thousands of rounds the sum of trees gives a very accurate log-odds. LightGBM grows trees leaf-wise using histograms of feature values, which makes it fast on large data.

Tuning: random search of 40 settings per version (num_leaves 15-255, min_data_in_leaf 20-2,000, feature_fraction, bagging_fraction, L1/L2 regularisation, min_gain_to_split), learning rate 0.05 with **early stopping** (stop when validation AUC has not improved for 100 rounds), then a final fit at learning rate 0.02. Chosen for B: 21 leaves, min_data_in_leaf 1,433, feature_fraction 0.64, bagging 0.65, 2,279 rounds. Small, heavily regularised trees won.

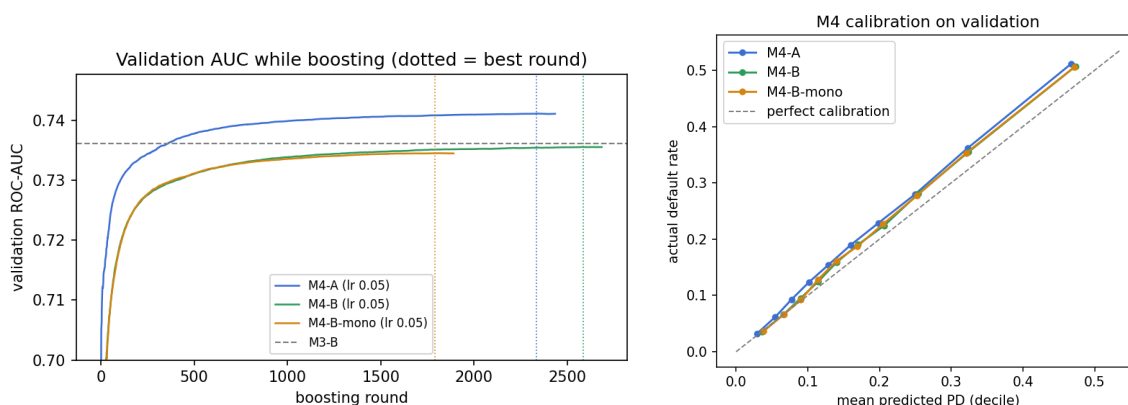
Results: validation ROC-AUC **0.7362** (B) / 0.7414 (A), Brier 0.1426: a clear +0.011 over logistic regression. Version B is only 0.005 behind A, so our features rebuild most of LendingClub's own risk opinion. **B-mono** adds monotonic constraints on 6 features and costs only 0.0008 AUC (0.7354).



M4: XGBoost — the robustness check

Why another booster: if a second, independent library gives the same result, the LightGBM result is not a fluke of one implementation. XGBoost grows trees level-wise (depth-limited) with the 'hist' method. Same search design: max_depth 3-10, min_child_weight, subsample, colsample_bytree, L1/L2, gamma; chosen depth 3 with ~2,600 rounds.

Results: validation ROC-AUC 0.7356 (B) vs 0.7362 for LightGBM: a tie (difference -0.0006, and the two models rank loans almost identically, Spearman 0.990). By the rule fixed in advance, ties go to the faster model: LightGBM trains about 2.3× faster (41 s vs 97 s per search fit). CatBoost was considered and skipped because two boosters already agreed.

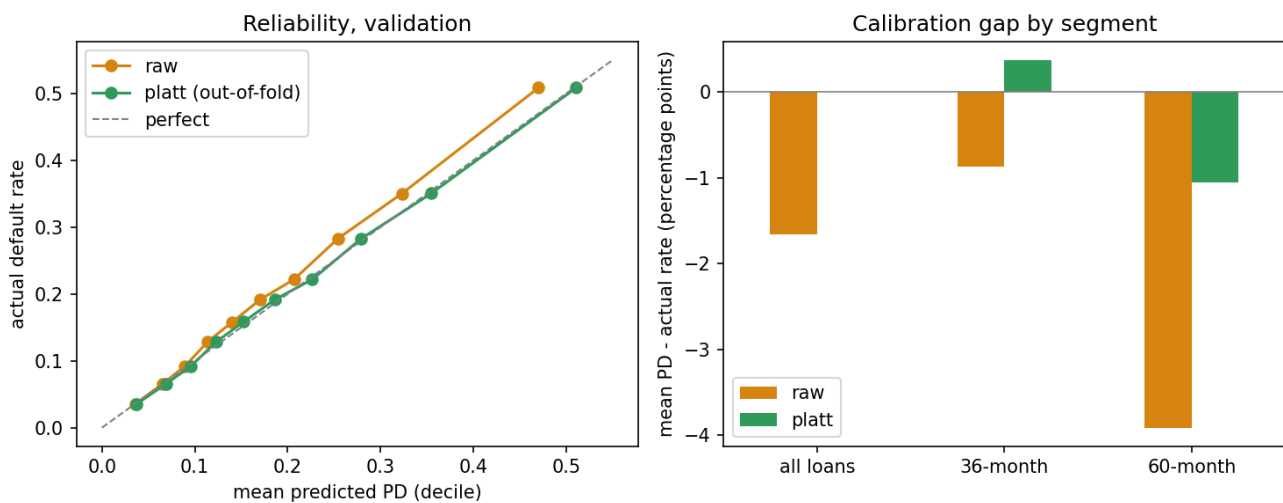


Finalising the model (models/final/, Stages 1-5)

Stage 1: monotonic constraints and calibration

Monotonic constraints force the model to respect credit logic: PD can only go **down** as FICO or income go up, and only **up** as DTI, term, inquiries or revolving utilisation go up. We tested 1,000 validation applicants by sweeping each feature while holding everything else fixed: the unconstrained model had **3,709 violations** (e.g. 99.9% of applicants had at least one income value where more income raised the PD), the constrained model **0**. Rule fixed beforehand: choose the constrained model if it costs less than 0.005 AUC; it cost 0.0008.

Calibration: the raw model said 18.7% on average while the validation default rate was 20.35% (drift from the 2012-14 training period). We compared no calibration, **Platt scaling** (a logistic regression on the logit of the PD: $PD_{cal} = 1 / (1 + e^{-(a \cdot \text{logit}(PD) + b)})$, 2 numbers) and **isotonic regression** (a step-shaped monotone mapping), each scored out-of-fold with 5-fold cross-fitting inside validation. Platt won the tie-break (Brier 0.14236, simpler, saved as plain JSON: a = 1.0412, b = 0.1723) and brought the mean PD to exactly 20.35%.



Stage 2: fairness and reason codes

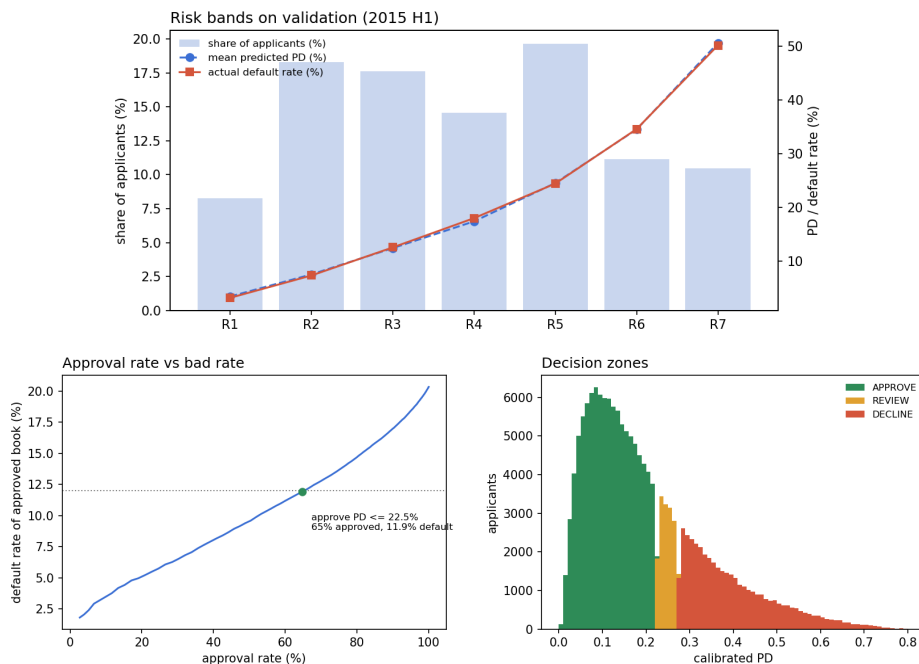
The data has no race, sex or age, so the main fairness risk is **geography as a proxy**: an area's default history can stand for the people who live there. Rule fixed beforehand: drop zip_code if it adds less than 0.002 AUC. It added only **+0.0004** (0.7354 with vs 0.7351 without), so it was dropped; without it, approvals shift toward the highest-default areas by +2.2 points, meaning zip had partly judged people by where they live. The final model, **M3-B-mono-nozip**, has 146 features, 0 monotonicity violations and was recalibrated (a = 1.0386, b = 0.1685).

State-level check: for 33 states with $\geq 1,000$ loans the calibration gap stayed within ± 2.8 points; LA, TN and OR were flagged for monitoring. Reason codes come from SHAP (next chapters); the most common decline reasons were loan term (60%), FICO (56%) and DTI (43%).

Stage 3: the decision layer

Loss model from the 66,928 defaulted training loans: EAD share (principal still owed at default ÷ funded) = 57.3% for 36-month and 70.5% for 60-month loans; LGD (share of that never recovered, after collection fees) = 88.7% / 88.8%. **Expected loss** = PD × loan amount × EAD share × LGD, e.g. a \$10,000 36-month loan with PD 20% → $0.2 \times 10,000 \times 0.573 \times 0.887 \approx \$1,015$.

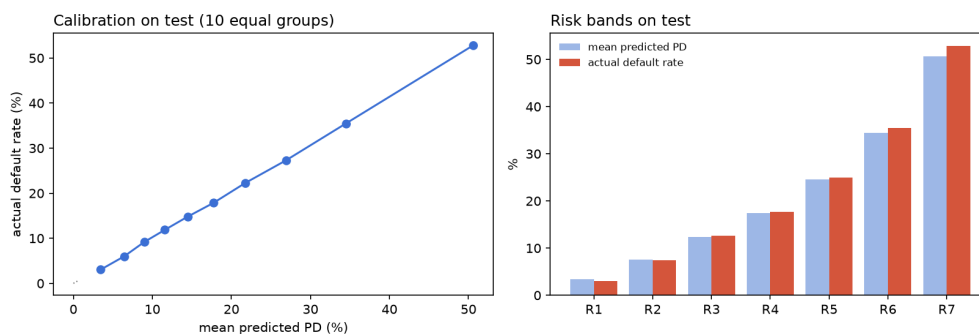
Risk bands R1-R7 at fixed PD cut-offs 5/10/15/20/30/40%; the actual default rate rises in every band (3.2% → 50.1%). **Thresholds** from two business targets written before looking: approve the largest group whose default rate is ≤ 12%, decline the riskiest group whose default rate is ≥ 40%. Result: **APPROVE** if PD ≤ 22.5% (64.7% of validation applicants, 11.9% default), **DECLINE** if PD ≥ 27.5% (25.5%, 40.1% default), **REVIEW** in between (9.8%). Expected loss back-tested at 0.94× the real loss.



Stages 4 and 5: the scoring package and the one-time test

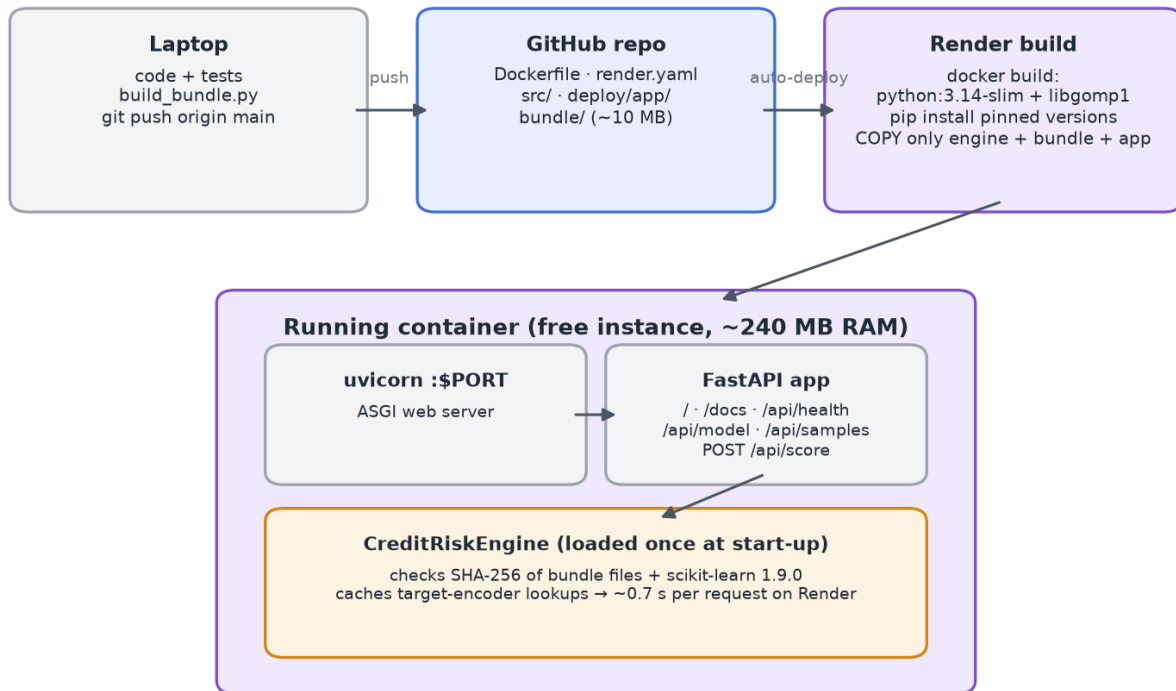
Stage 4 packaged the preprocessor, model, calibrator and policy into a checksummed bundle with *CreditRiskEngine*. Proof of correctness: the original raw CSV rows of 2,000 validation loans reproduce Stage 3's PD, band, decision and expected loss with a difference of exactly 0.0.

Stage 5 opened the sealed 2015 H2 test set once. The pass criteria were committed to git before opening it. Results: ROC-AUC **0.7465** (validation 0.7351), KS 0.360, Brier 0.1388, mean PD 19.6% vs 20.1% actual; LendingClub's interest rate ranks the same loans at 0.7148. 7 of 8 criteria met; the miss is the 60-month calibration gap (-2.5 points, documented, not fixed, because changing the model after seeing the test would make the test meaningless). On 2016-2018 stress data the AUC stays 0.70-0.72.



7. Deployment

Deployment: GitHub → Render → Docker → FastAPI



Free tier: the service sleeps after 15 idle minutes; the next visit wakes it in ~50 s.

FastAPI (deploy/app/main.py) is a Python web framework. It exposes *GET* / (the web page), */api/health*, */api/model* (model card), */api/samples* (six real 2015 applicants) and *POST /api/score*, and generates interactive documentation at */docs* automatically from the Pydantic request model. The engine is loaded once at start-up (FastAPI lifespan), so each request only scores, it does not reload files.

Docker packages the app with its exact environment: the Dockerfile starts from python:3.14-slim, installs libgomp1 (OpenMP, needed by LightGBM) and the pinned library versions (scikit-learn 1.9.0 must match the version that fitted the preprocessor), and copies only the engine code, the 10 MB bundle and the app. No training data goes into the image. **Render** builds that image from GitHub using render.yaml (a 'Blueprint') and runs it as a free web service; every push to main redeploys automatically. The app uses ~240 MB of RAM, inside the free 512 MB limit.

Problems solved during deployment

- **Hugging Face Spaces** was the first plan, but Docker Spaces now need a paid plan (HTTP 402), so we moved to Render.
- **Line endings:** Git on Windows can rewrite the model text file and break its checksum, so the bundle is marked binary in .gitattributes.
- **Speed:** scoring took 2.5-3.4 s on Render because scikit-learn's target encoder rebuilds a lookup of 143,539 job titles on every call. Caching it as a dictionary at start-up gave identical outputs (tested to 0.0 difference) and ~0.7 s per request.
- **Cold starts:** the free service sleeps after 15 idle minutes; the page shows a 'waking up' message.

Guardrails added after two independent audits (v1.1 → v1.3)

A friend's AI agent audited the live app as a black box (~540 requests in total). It found that the model itself behaved sensibly on the main features, but that the product around it approved absurd inputs. Because the test set was already used, we did **not retrain**; instead we added a policy layer around the frozen model and measured its effect on the test set.

Audit finding	Fix (version)	Why
FICO 300 was approved	REFER if FICO < 660, DTI > 40, loan outside \$1k-\$35k, income < \$3k... (v1.1)	LendingClub never lent below 660, so the model never saw such people; its PD there means nothing.
Empty application approved	10 core fields required (v1.1)	A PD built on imputed values only is not a decision.
Blank loan title gave a bonus	Title filled from purpose, like LendingClub's form (v1.1)	A rare 'no title' group had a low default rate.
Bureau data missing → approved	APPROVE becomes REVIEW if < 50% of bureau fields given (v1.1)	Imputed medians look 'average-safe'.
\$35k loan on \$3k income approved	REFER if loan > 50% of annual income (v1.2)	Training max loan-to-income was exactly 0.50.
REFER still showed a PD	Score hidden on REFER (v1.2)	Out of range, the number is not valid.
Server errors (500) on bad input	Strict number parsing, whole-number counts (v1.2)	Clean 422 messages instead.
Made-up job title lowered PD	Unseen titles treated as 'not provided' (v1.3)	Unknowns got the overall average (17%).

Effect on the sealed test set: the v1.2 rules change 123 of 212,801 decisions (0.06%). The v1.3 title rule touches 19% of loans (real but unseen job titles) and 5,833 decisions; ROC-AUC 0.7465 → 0.7456, mean PD 20.3% vs 20.1% actual. Not fixed (would need retraining): delinquency and credit-age effects learned from approved-only data, and state as an input.

8. SHAP: explaining each prediction

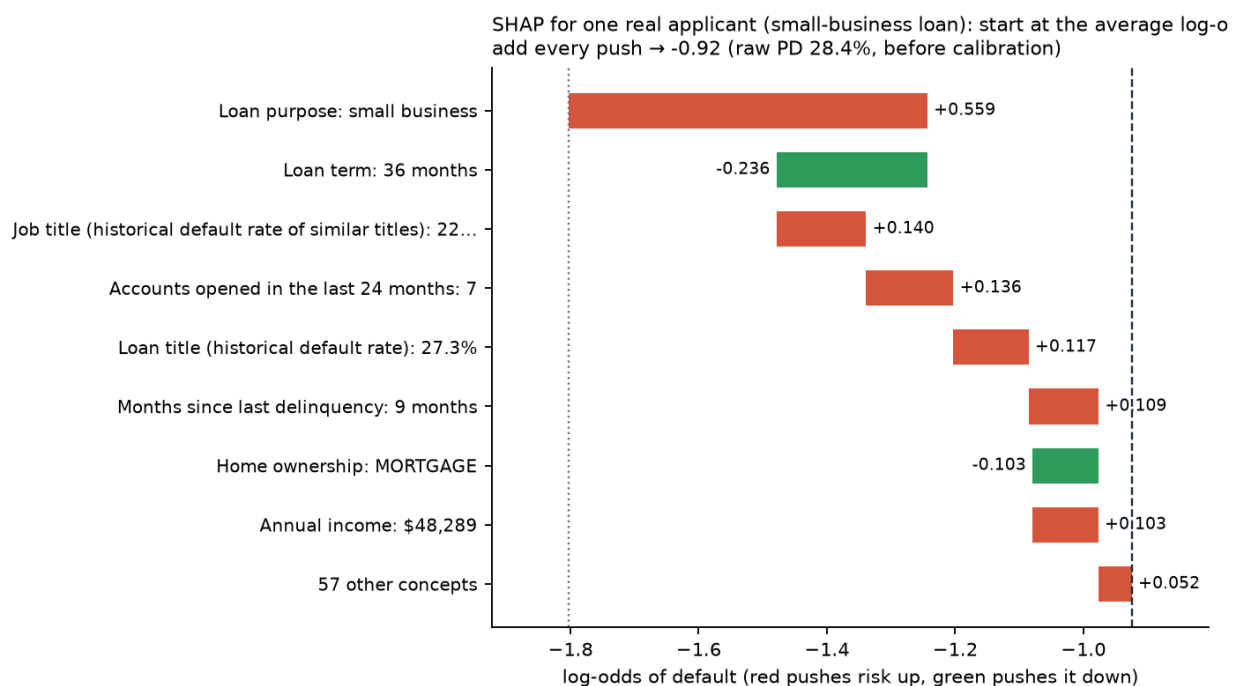
The question SHAP answers: for one applicant, how much did each feature push the prediction up or down compared with an average applicant? A model that says 'PD = 38%' is not enough in lending; the applicant has the right to know the main reasons, and the lender needs to check that the reasons make sense.

The idea comes from game theory (Shapley values). Think of the features as players in a team and the prediction as the team's score. A feature's Shapley value is its average marginal contribution over all possible orders in which the features could join the team. This is the only way of splitting the credit that is fair in a precise mathematical sense (efficiency, symmetry, dummy and additivity properties).

$$\text{prediction}(x) = \text{base value} + \phi_1 + \phi_2 + \dots + \phi_n$$

Here the base value is the model's average output, and ϕ_i is the SHAP value of feature i for this applicant. The most important property is **additivity**: the SHAP values add up exactly to the prediction. We checked this on all 162,745 validation loans: the largest difference was 3.3×10^{-14} , i.e. exact. For LightGBM the values are in **log-odds** (the model's raw output before the sigmoid), so they add up to the log-odds of default; positive values push risk up, negative values push it down.

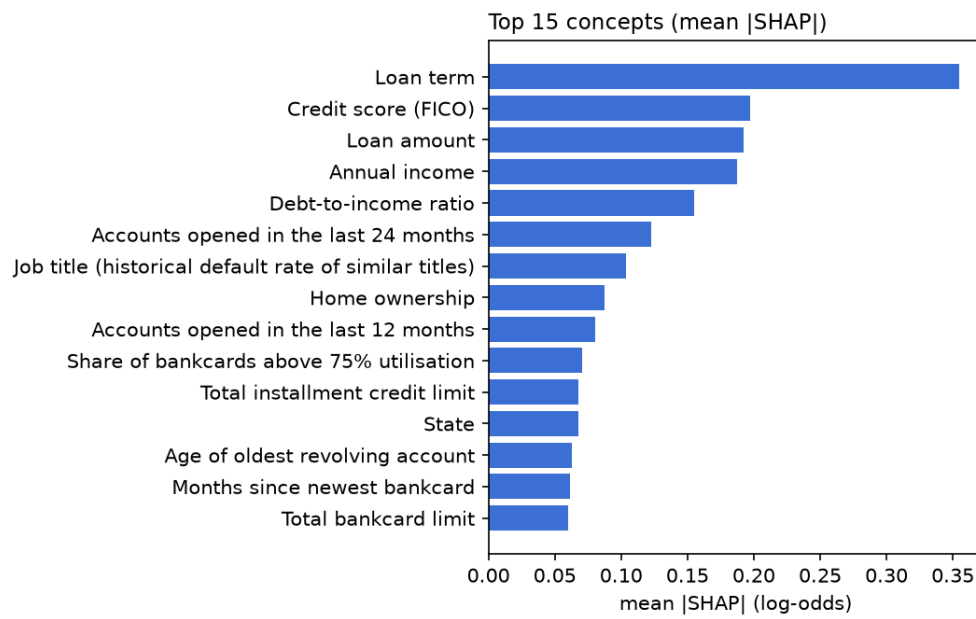
How we computed it: exact Shapley values are expensive in general, but for tree models there is a fast exact algorithm (TreeSHAP). LightGBM has it built in: `booster.predict(X, pred_contrib=True)` returns one value per feature plus the base value. It takes about 11 ms for one applicant, so it runs live on every request.



From SHAP values to reason codes

- **Group into concepts:** the model has 146 columns, but several describe one idea (all `addr_state_*` dummies are 'State', a missing flag belongs to its feature). Because SHAP is additive, a concept's value is the sum of its columns' values (65 concepts).
- **Rank:** the 4 largest positive concepts are the risk reasons (the usual number in US adverse-action notices); the 2 largest negative ones are strengths.
- **Describe with the applicant's own value:** 'Credit score (FICO): 660', 'Debt-to-income ratio: 37.0%', 'Loan term: 60 months' (`src/modeling/reason_codes.py`).
- **Guard (v1.2):** a derogatory record or a missing value is never shown as a strength, because the model learned some of those effects backwards from approved-only data.

Global view: averaging |SHAP| over all applicants gives feature importance. The top concepts were loan term, FICO, DTI, job title, accounts opened in the last 24 months, loan amount and income.



9. Evaluation metrics: formulas and why each was used

Every model was scored with the same function (src/modeling/evaluation.py). Notation: $y = 1$ for default, 0 for repaid; p = predicted probability of default; N = number of loans. For a threshold t , a loan is 'predicted default' if $p \geq t$, which gives the confusion matrix: TP (defaulter flagged), FP (good borrower flagged), TN, FN.

Why not accuracy?

$$\text{Accuracy} = (TP + TN) / N$$

With 20% defaults, predicting 'repaid' for everyone is 80% accurate and useless. Logistic regression at threshold 0.5 was 80% accurate but caught only 3% of defaulters. Accuracy depends on one arbitrary threshold and hides the minority class, so it is reported only as a warning, never used to choose models.

ROC-AUC (main metric)

$TPR = TP / (TP + FN)$ · $FPR = FP / (FP + TN)$ · AUC = area under the curve of TPR vs FPR over all thresholds

$$\text{Equivalent: } AUC = P(p_{\text{random defaulter}} > p_{\text{random good borrower}})$$

Meaning: the probability that the model gives a randomly chosen defaulter a higher PD than a randomly chosen repaid borrower. 0.5 = random, 1.0 = perfect. **Why:** lending is a ranking problem (approve the safest first); AUC measures ranking quality independent of any threshold and of the default rate, so it is comparable across periods. Typical consumer-credit models on application data score 0.70-0.80, so 0.746 is realistic, not suspicious.

PR-AUC (average precision)

$$\text{Precision} = TP / (TP + FP) \cdot \text{Recall} = TPR \cdot AP = \sum_k (R_k - R_{k-1}) \cdot P_k$$

Why: it focuses on the minority class (defaulters). Its baseline is the default rate (0.20), not 0.5, so our 0.43 on test is about 2.2× better than random. Useful as a second view of ranking on imbalanced data.

KS statistic (Kolmogorov-Smirnov)

$$KS = \max_t |F_{\text{default}}(t) - F_{\text{repaid}}(t)| = \max_t (TPR(t) - FPR(t))$$

Meaning: the largest gap between the score distributions of defaulters and repaid borrowers. **Why:** it is the traditional metric in banking credit scorecards, so lenders and interviewers from banks recognise it. 0.36 on test is good for application scoring.

Brier score

$$\text{Brier} = (1/N) \sum_i (p_i - y_i)^2$$

Meaning: mean squared error of the probabilities; lower is better. It rewards both ranking and correct probability levels. **Why:** our PD feeds expected loss in dollars, so the probability itself must be right, not just the order. Used to choose the calibration method.

Log loss (binary cross-entropy)

$$\text{LogLoss} = -(1/N) \sum_i [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

Why: it is the loss the boosting models actually minimise, and it punishes confident wrong predictions heavily. Watching it on validation shows overfitting.

Calibration gap and ECE

$$\text{Calibration gap} = \text{mean}(p) - \text{mean}(y) \cdot ECE = \sum_b (n_b/N) \cdot |\text{mean}(p)_b - \text{mean}(y)_b|$$

Meaning: the gap checks the overall level (we got -0.4 points on test); ECE (expected calibration error) splits loans into 10 equal groups by PD and averages the absolute gap per group (0.6% on test). **Why:** 'PD = 20%' must mean 20 in 100 default, otherwise the decision thresholds and expected loss are wrong.

Approval rate vs bad rate (business metric)

for approval rate a : approve the $a\%$ lowest PDs $\rightarrow$ bad rate(a) = defaults among approved / approved

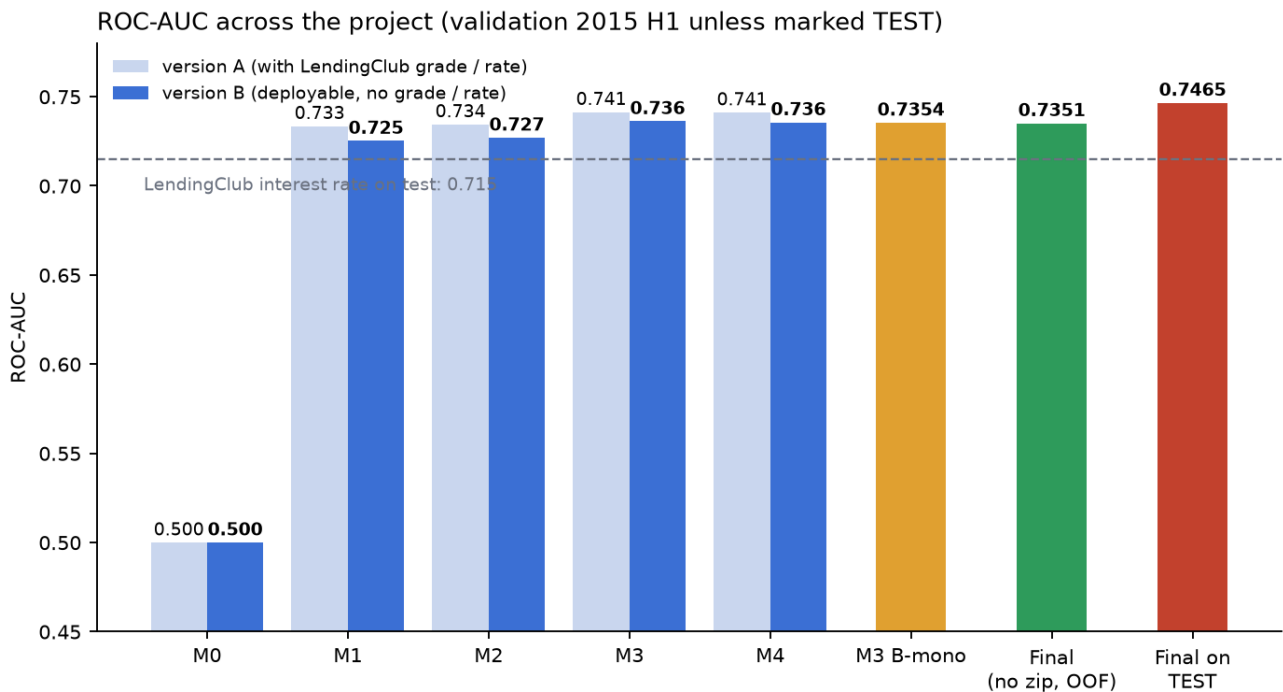
Why: this is how a lender reads a model: 'if we approve 65%, what default rate do we get?'. It was used to pick the approve threshold (approved book $\leq 12\%$ default).

Expected loss and its back-test

$$EL = PD \times EAD \times LGD \cdot EAD = \text{amount} \times EAD \text{ share} \cdot \text{back-test ratio} = \Sigma EL / \Sigma \text{ realised loss}$$

Why: it turns the probability into money. A ratio near 1.0 (0.94 on validation, 0.89 on test) means the dollar forecasts are trustworthy at portfolio level.

10. Results across all models



Model / step	ROC-AUC	Note
M0 Dummy	0.500	floor: same PD for everyone
M1 Logistic Regression (B)	0.7252	linear scorecard, log+scale, C = 0.001
M2 Random Forest (B)	0.7270	+0.002, overfits (train 0.855)
M3 LightGBM (B)	0.7362	+0.011 over logistic → chosen
M4 XGBoost (B)	0.7356	tie with M3 (Spearman 0.990)
M3 B-mono	0.7354	monotonic constraints, 0 violations, cost 0.0008
Final M3-B-mono-nozip	0.7351	zip dropped, recalibrated (out-of-fold validation)
Final on sealed TEST	0.7465	212,801 loans, opened once; LendingClub rate 0.7148
Stress 2016 / 2017 / 2018	0.719 / 0.711 / 0.704	ranking only (biased labels)

Version A (with LendingClub's grade and rate) is always 0.005-0.008 higher on validation, which means our features rebuild most of LendingClub's own risk opinion without using it. The test score is higher than validation for every model, including LendingClub's own interest rate (0.704 → 0.715), so the 2015 H2 loans were simply easier to rank; it is not leakage.

11. Interview preparation: questions from the resume

Every number on the resume, with the question an interviewer is likely to ask and a short answer you can give.

Resume: "Built an end-to-end credit-risk engine on 1.35M LendingClub loans"

Q: Why 1.35M and not 2.26M?

Only loans with a final outcome (Fully Paid or Charged Off) can be labelled. 912,602 loans were still Current or Late at the 2018 snapshot, so they were dropped. 1,348,099 remain, 20% defaults.

Q: What was in the leakage audit of 151 columns?

Each column was classified by 'is it known when the application arrives?'. 42 post-loan columns were removed (payments, recoveries, last FICO, settlement, hardship...), plus target, ids and admin fields: 151 → 103 candidates. The highest single-feature AUC afterwards was 0.674 (interest rate), below a 0.80 leak flag.

Q: What is an out-of-time split and why use it?

Train on loans issued 2012-08 to 2014-12, validate on 2015 H1, test on 2015 H2. A random split lets the model see the future period during training and overstates quality; credit models are used on future applicants, so they must be tested that way.

Q: What were the 72 audits?

Target alignment, temporal ordering, leakage, data hygiene (no NaN/inf), consistency of features across splits, persistence of the saved pipeline, and engineering checks, plus a fresh-process test that raw rows reproduce the exported features exactly.

Resume: "Benchmarked 5 models; monotonic, calibrated LightGBM; ROC-AUC 0.746 vs 0.715"

Q: Which 5 models and why that order?

Dummy (floor), Logistic Regression (simple explainable baseline), Random Forest (non-linearity), LightGBM (boosting), XGBoost (a second booster to confirm). Each had to beat the previous one.

Q: Why LightGBM over XGBoost?

They tied (0.7362 vs 0.7356 validation, rankings 99% correlated). By a rule fixed beforehand, a tie goes to the faster model; LightGBM was 2.3× faster.

Q: What does 'never sees the lender's grade or rate' mean?

Grade, sub-grade, interest rate and instalment are LendingClub's own risk opinion, set after its review. A lender building its own engine would not have them, so the deployable version B excludes them. Version A with them scored only ~0.005 higher.

Q: What are monotonic constraints?

Rules that force PD to move one way: down with FICO and income, up with DTI, term, inquiries and utilisation. Without them the model had 3,709 logic violations on 1,000 test applicants; with them 0, at a cost of 0.0008 AUC.

Q: What does calibrated mean and how?

The predicted PD matches the real default rate. The raw model said 18.7% on average vs 20.35% actual (drift since training). Platt scaling (2-parameter logistic on the logit) fixed it, chosen over isotonic by out-of-fold Brier score.

Q: Is 0.746 good? Why is test higher than validation?

Consumer credit models on application data usually score 0.70-0.80. Test was higher for every model including LendingClub's own rate (0.704 → 0.715), so 2015 H2 was easier to rank, not leakage. The test set was sealed in code and the criteria committed to git before it was opened once.

Q: What is the 0.715?

The ROC-AUC you get by ranking the same test loans by LendingClub's interest rate, i.e. LendingClub's own pricing. Our model beats it without using it. Caveat: both are measured only on loans LendingClub approved.

Resume: "Decision layer: risk bands, thresholds, expected loss ($PD \times EAD \times LGD$), SHAP; 11.6% vs 20.1%; 13.6% → 6.2%"

Q: How were the thresholds chosen?

From two business targets written before looking at results, searched on validation with out-of-fold PDs: approve the largest group with default rate $\leq 12\%$ ($PD \leq 22.5\%$), decline the riskiest group with default rate $\geq 40\%$ ($PD \geq 27.5\%$), review in between.

Q: Explain $PD \times EAD \times LGD$.

PD = chance of default. EAD = how much is still owed when it happens (57% of the loan for 36-month, 71% for 60-month, from defaulted training loans). LGD = share of that never recovered (~89%). Their product is the expected loss; on validation it matched real losses at 0.94x.

Q: What do 11.6% vs 20.1% and 13.6% → 6.2% mean?

On the test set, the loans the engine approves (66.8% of applicants) defaulted at 11.6%, against 20.1% for all loans LendingClub funded. Realised loss per dollar lent falls from 13.6% (approve everyone) to 6.2% (approve only APPROVE). Caveat: lost interest from rejected good borrowers is not counted.

Q: What is SHAP?

Shapley values from game theory: each feature's fair share of the difference between this prediction and the average. They add up exactly to the model output (checked to 10^{-14}). We group them into 65 concepts and show the top 4 as plain-language reasons.

Resume: "FastAPI + Docker; fairness ablation; audit-driven guardrails"

Q: Walk me through a request.

Browser → Render → Docker container → uvicorn → FastAPI → validation and guardrails → CreditRiskEngine (preprocessor → LightGBM → Platt → policy → SHAP) → JSON. ~0.7 s on the free server.

Q: What is the checksummed bundle?

The four model files plus a manifest with SHA-256 hashes and library versions. The engine refuses to start if a file changed or scikit-learn is not 1.9.0, so the live app scores exactly like the tested model.

Q: What was the fairness ablation?

Train with and without zip code. It added only +0.0004 AUC, below the 0.002 rule, so it was dropped because it can act as a proxy for where people live. Approvals then shifted +2.2 points toward high-default areas.

Q: What guardrails and why didn't you retrain?

Two black-box audits found absurd approvals (FICO 300, \$35k on \$3k income, empty forms). Fix: REFER out-of-range applicants with no score shown, required fields, strict validation, affordability rule, no auto-approval without bureau data, unknown titles treated as missing. No retraining because the test set was already used; a retrained model could not be honestly re-tested. The v1.2 rules changed 0.06% of test decisions.

Likely follow-up questions

Q: Biggest limitation?

Selection bias: trained only on approved loans, so it never saw people LendingClub rejected (fixed partly with REFER rules). Also 60-month loans are under-predicted by 2.5 points on test, and delinquency effects are weak.

Q: How would you improve it?

Reject inference using the rejected-applications file; a term-specific calibrator on newer data; retrain without free-text titles and with more monotonic constraints; monitoring for drift (PSI) and periodic recalibration.

Q: Why not deep learning?

On tabular data with ~150 features, gradient-boosted trees are the state of the art, train fast on CPU and give exact SHAP values; neural nets rarely beat them here and are harder to constrain and explain.

Q: How did you handle class imbalance?

No resampling: 20% defaults is enough signal (33k+ defaults per split). `class_weight` was tested and rejected because it distorts probabilities, and we need calibrated PDs.

Q: What would you monitor in production?

Input drift (PSI per feature), the default rate of approved loans vs predicted, calibration by segment (term, state), approval rate, REFER rate and latency.

Numbers to remember: 2.26M loans → 1.35M resolved (20% default) · 151 → 103 → 190 → 146 features · 42 leakage columns · 72 audits · train 388,124 / val 162,745 / test 212,801 · ROC-AUC 0.736 (val) / 0.746 (test) vs 0.715 · 3,709 → 0 violations · zip +0.0004 → dropped · Platt $a \approx 1.04$ · approve PD $\leq 22.5\%$, decline $\geq 27.5\%$ · 66.8% approved at 11.6% default · loss 13.6% → 6.2% · EAD 57% / 71%, LGD ~89% · ~0.7 s per request · guardrails changed 0.06% of test decisions.